

Engenharia de Software

Licenciatura em Engenharia Informática

Saúde Prime++

Relatório de Desenvolvimento de Software

Aplicação de Gestão Clínica Multi-Perfil (Flutter e Supabase)

Docente

Professor José Duarte

Grupo 1

Felipe Castilho a047152
Juliana Moreira a047188
Marta Vieira a046756
Ricardo Dias a047068
Rodrigo Esteves a048074

Universidade da Maia

Junho 2026

Índice

1. Sumário Executivo 4
    Destaques 4
2. Visão do Produto 5
3. Entregáveis do Projeto 5
4. Engenharia de Requisitos 6
    4.1 Requisitos 6
    4.2 Modelo de Casos de Uso 7
    4.3 Modelo de Domínio 8
5. User Stories e Priorização (MoSCoW) 9
6. Funcionalidades Implementadas 10
    6.1 Autenticação e Controlo de Acesso 10
    6.2 Portal do Paciente 10
    6.3 Portal do Médico 10
    6.4 Portal da Receção 10
    6.5 Portal do Admin 10
    6.6 Notificações: Emails e SMS 11
    6.7 Edge Functions e Funcionalidades Assíncronas 11
7. Arquitetura e Design 12
    7.1 Vista Lógica (Diagrama de Pacotes) 12
    7.2 Vista de Implementação 13
    7.3 Vista de Deployment 14
    7.4 Stack Tecnológica 14
8. Evolução por Versões e Incrementos 15
9. Processo, Iterações e Uso do Board 15
    9.1 Iteração 1: Frontend e Fundações 15
    9.2 Iteração 2: Integração de Backend 15
    9.3 Iteração 3: Edge Functions, Notificações e Testes 15
10. Testes e Qualidade 16
    10.1 Abordagem 16
11. Fluxo de Trabalho no GitHub 17
12. Dificuldades e Como as superámos 17
    12.1 Não saber por onde começar a base de dados 17
    12.2 Políticas de segurança (Row-Level Security) 18
    12.3 Migrações e a passagem de dados mock para dados reais 18
    12.4 Fila virtual em tempo real 18
    12.5 Testes automáticos para Flutter e Supabase 18
    12.6 Notificações assíncronas e segredos (email e SMS) 19

12.7 Trabalho em equipa numa única base de código 19
13. Uso de Inteligência Artificial 19
14. Contribuição do Grupo 19
15. Conclusão 20

1. Sumário Executivo

A Saúde Prime++ é uma aplicação de gestão clínica multiperfil que centraliza, num único produto, o agendamento de consultas, o acesso a documentos clínicos, uma fila virtual de atendimento e as operações entre os quatro tipos de utilizadores de uma clínica: pacientes, médicos, rececionistas e administradores.

O sistema foi desenvolvido em Flutter (cliente cross-platform para Android, iOS e Web) com Supabase como Backend-as-a-Service: PostgreSQL, autenticação, armazenamento de objetos e Edge Functions. O controlo de acesso é aplicado diretamente na base de dados, através de políticas Row-Level Security (RLS), dispensando um servidor de aplicação dedicado.

O desenvolvimento decorreu de forma incremental ao longo de quatro versões (v0.0 a v3.0) e três iterações, mapeadas nos quatro incrementos de produto exigidos. Todos os entregáveis da unidade curricular foram concluídos e estão no repositório: visão de produto, casos de uso, user stories, modelo de domínio, mockups, testes de aceitação, uso do board nas quatro iterações, arquitetura lógica e física, testes automatizados, integração com backend e bom uso do GitHub.

As 15 user stories planeadas foram implementadas, incluindo a totalidade dos requisitos Must Have e as funcionalidades de maior esforço (newsletter, recuperação de password e notificações de resultados). O produto entrega notificações reais por email (Gmail SMTP) e por SMS (Twilio), geração automática de faturas em PDF, fila virtual com tempo de espera, registo de auditoria e exportação de relatórios.

Destaques

Indicador	Valor
Versões / Incrementos entregues	4 (v0.0, v1.0, v2.0, v3.0)
User Stories implementadas	15 de 15 (todas)
Portais aplicacionais	4 (Paciente, Médico, Receção, Admin)
Edge Functions (Deno)	6
Canais de notificação	Email (Gmail SMTP) e SMS (Twilio)

2. Visão do Produto

Para pessoas que precisam ou procuram análises e consultas especializadas, a Saúde Prime++ é uma aplicação de gestão clínica multiperfil que centraliza o agendamento de consultas, o acesso a documentos clínicos, a fila virtual e as operações entre pacientes, médicos, rececionistas e administradores.

"Descarregue e atualize já a versão mais recente da sua saúde com a Clínica Médica Saúde Prime++"

A aplicação é cross-platform (Android, iOS e Web) a partir de uma única base de código, com Supabase como Backend-as-a-Service e cumprimento dos princípios de privacidade e proteção de dados (RGPD). O controlo de acesso é aplicado ao nível da base de dados via RLS, e o desenho prevê integração futura com sistemas externos (laboratórios, seguradoras e plataformas de pagamento).

3. Entregáveis do Projeto

O projeto foi avaliado por um conjunto de entregáveis distribuídos ao longo do semestre, cobrindo engenharia de requisitos, arquitetura, incrementos de produto, qualidade e práticas de engenharia. A tabela seguinte resume cada entregável, os respetivos pontos-chave e o estado. Todos os artefactos estão versionados no repositório GitHub.

Entregável	Pontos-chave	Peso	Estado
Product Vision Statement	Conceito de alto nível, claro e conciso	1,0%	Concluído
Use Cases	Casos de uso essenciais, correção UML e descrição textual	3,0%	Concluído
User Stories	Quem, o quê, porquê; independentes e com valor	6,0%	Concluído
Domain Model	Domínio bem modelado, correção UML	3,0%	Concluído
UI Mockups	Mockups representativos e bem desenhados	2,0%	Concluído
Acceptance Tests	Estilo Given/When/Then, Gherkin, em inglês	2,0%	Concluído
Board well used (#0 a #3)	Kanban e backlog estimado, atribuído e monitorizado	5,0%	Concluído

Entregável	Pontos-chave	Peso	Estado
Product Increment #0	Protótipo vertical	3,0%	Concluído
Product Increment #1	Âmbito definido, release usável, valor claro	3,0%	Concluído
Product Increment #2	Mesmos critérios do incremento #1	3,0%	Concluído
Product Increment #3	Mesmos critérios do incremento #1	3,0%	Concluído
Logical Architecture	Diagrama de Pacotes UML e descrição	2,0%	Concluído
Physical Architecture	Diagramas de Componentes e Deployment UML	4,0%	Concluído
Test Cases (Automated)	Testes unitários e de aceitação automatizados	2,0%	Concluído
GitHub Flow / Usage	Mensagens claras, bom uso de tags e branches	1,0%	Concluído
Business Value	Valor percebido e funcionalidades para produção	5,0%	Concluído
Technical Maturity	Qualidade técnica e decisões tecnológicas corretas	5,0%	Concluído
Integration with Backend	Integração com BD e serviços externos / APIs	5,0%	Concluído
Presentation and Demo	Consistência, clareza e bom suporte visual	2,0%	Concluído

4. Engenharia de Requisitos

4.1 Requisitos

Os requisitos foram organizados em funcionais (autenticação e gestão de utilizadores, gestão de pacientes e histórico, agendamento, documentos clínicos, fila virtual e gestão de horários) e não funcionais (performance, segurança, usabilidade, fiabilidade, escalabilidade, manutenção e portabilidade). Destacam-se: resposta inferior a 2 segundos, encriptação dos dados dos pacientes, níveis de acesso por role e acesso web sem instalação.

4.2 Modelo de Casos de Uso

O sistema foi modelado num caso de uso geral (Gestão Integrada Saúde Prime++) decomposto em seis casos de uso principais: Autenticação e Gestão de Perfil, Gestão de Pacientes e Histórico, Agendamento de Consultas, Acesso e Transferência de Documentos Clínicos, Sistema de Fila Virtual e Gestão de Horários.

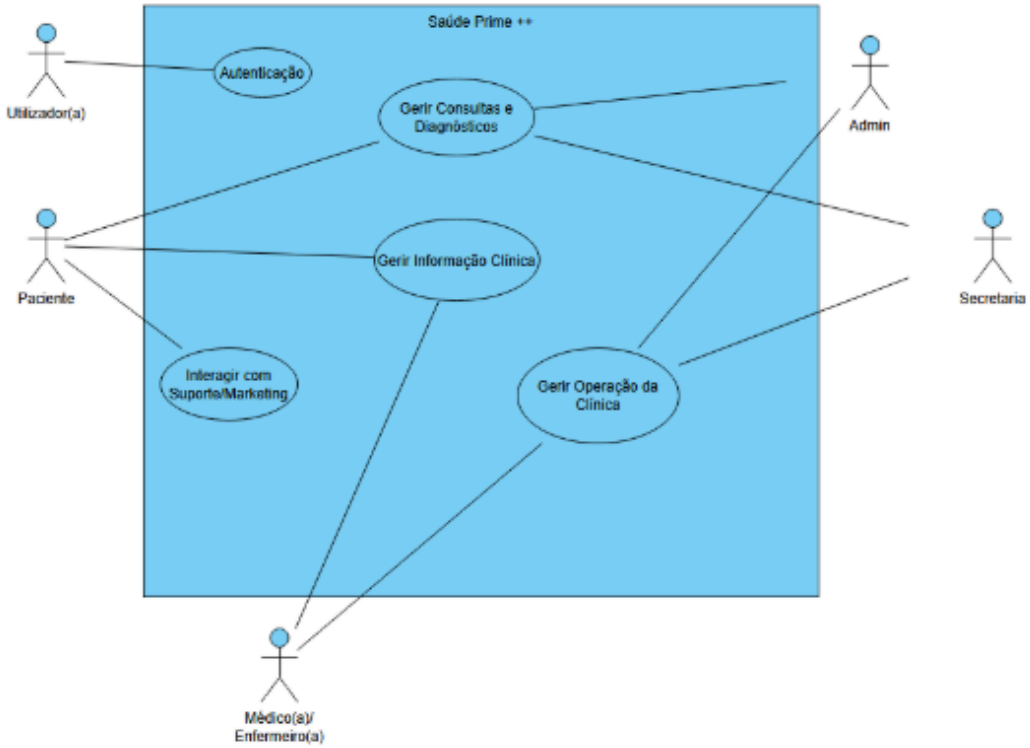


Figura 1: Diagrama de Casos de Uso Geral

4.3 Modelo de Domínio

O modelo de domínio centra-se na entidade User (paciente), da qual Doctor herda atributos específicos (número de licença, especialidade). A entidade central de operação é a Appointment, que liga paciente e médico, está associada a um Appointment_Type e gera automaticamente uma Invoice. Os utilizadores podem ter vários MedicalRecord (com PDFs), estar ligados a uma Insurance_Company, e os médicos definem Schedule_Block para períodos de indisponibilidade.

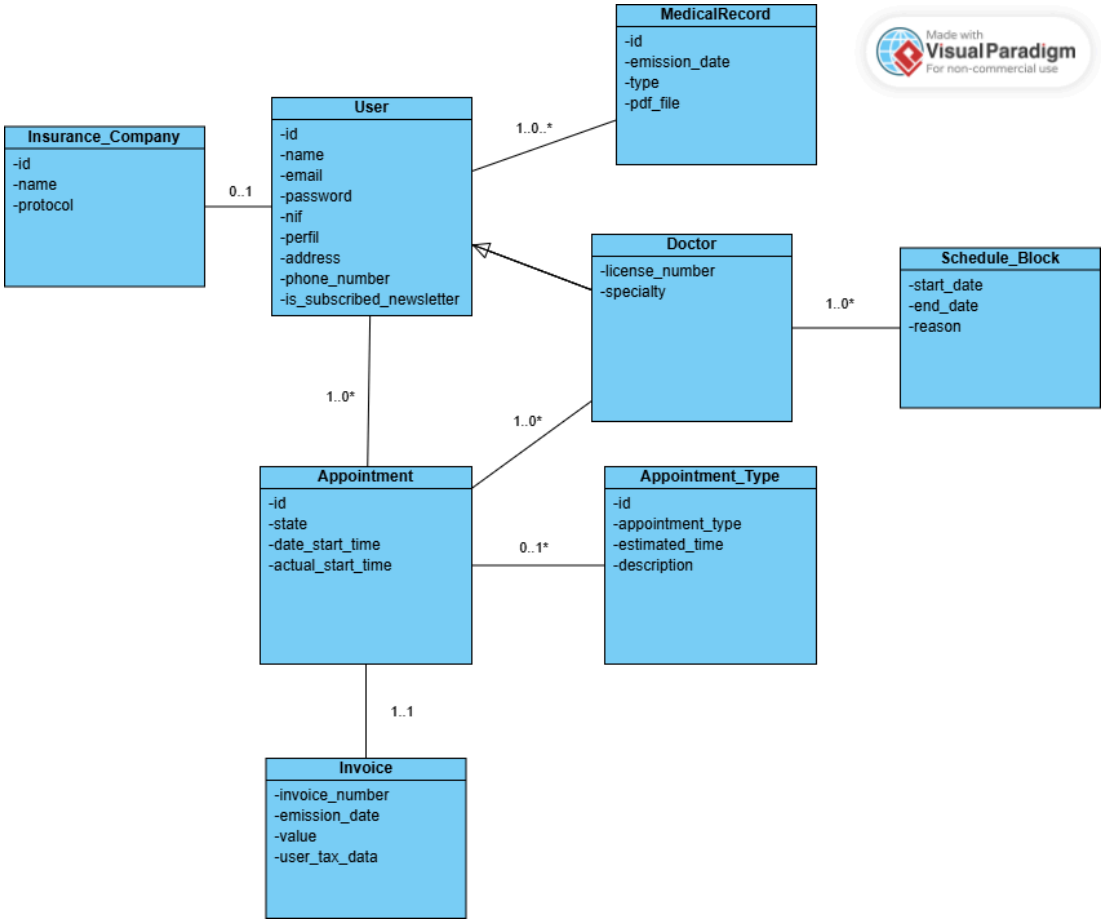


Figura 2: Modelo de Domínio (Diagrama de Classes UML)

5. User Stories e Priorização (MoSCoW)

Foram definidas 15 user stories, priorizadas por MoSCoW e estimadas em esforço (S/M/L/XL). Ao contrário do âmbito inicial, em que algumas histórias de esforço XL estavam classificadas como Will Not Have, a equipa conseguiu implementar a totalidade das 15 user stories, incluindo newsletter, recuperação de password e notificações de resultados.

ID	User Story	Esforço	MoSCoW	Estado
US1	Criar conta com dados básicos	M	Must	Concluída
US2	Atualizar contacto / morada no perfil	S	Must	Concluída
US3	Histórico cronológico de consultas e exames	M	Could	Concluída
US4	Pesquisar médicos por especialidade	S	Should	Concluída
US5	Subscrever newsletter semanal	XL	Won't	Concluída
US6	Médico marca indisponibilidade na agenda	M	Must	Concluída
US7	Marcar consulta (com locking)	L	Must	Concluída
US8	Reagendar consulta marcada	L	Should	Concluída
US9	Descarregar fatura da consulta	L	Must	Concluída
US10	Atribuir níveis de acesso (Admin)	L	Must	Concluída
US11	Ver tempo médio de espera da senha	L	Could	Concluída
US12	Exportar relatórios para seguradoras	M	Should	Concluída
US13	Redefinir password via email	XL	Won't	Concluída
US14	Cancelar consulta com antecedência	XL	Won't	Concluída

ID	User Story	Esforço	MoSCoW	Estado
US15	Notificação de novos resultados	XL	Won't	Concluída

6. Funcionalidades Implementadas

Esta secção detalha o que foi efetivamente construído e ligado ao backend real, organizado por portal e por funcionalidades transversais.

6.1 Autenticação e Controlo de Acesso

- Registo, login e logout com Supabase Auth (JWT) e encaminhamento por role.
- Recuperação e alteração de password por email (fluxo de reset).
- Quatro roles (Paciente, Médico, Rececionista, Admin) com permissões aplicadas por políticas RLS na base de dados.
- Prevenção de criação de utilizadores com dados duplicados e validações de registo.

6.2 Portal do Paciente

- Dashboard, agendamento de consultas em três passos, reagendamento e cancelamento.
- Histórico clínico cronológico e acesso aos seus documentos (download de PDFs).
- Fila virtual com senha e tempo médio de espera estimado.
- Centro de notificações e preferências (email, SMS, push, lembrete 24h, newsletter).
- Edição de perfil e eliminação da própria conta.

6.3 Portal do Médico

- Agenda e gestão dinâmica de horários; bloqueio de períodos de indisponibilidade.
- Registo de consulta (marcar como realizada) e pesquisa de pacientes.
- Envio de documentos clínicos para o paciente (upload para Storage).
- Gestão da fila virtual (chamar próximo, registar ausência).

6.4 Portal da Receção

- Gestão de pacientes (criar, editar, ver dados e consultas) com calendário de data de nascimento.
- Agendamento, alteração e cancelamento de consultas em nome do paciente.
- Controlo da fila virtual e definições; filtros por consultas eliminadas / suspensas.

6.5 Portal do Admin

- Gestão de utilizadores e alteração de permissões; criação de médicos com especialidade.
- Suspensão de médicos e alteração de especialidades.
- Bloqueio de marcações (blocked slots) e gestão de agendas.
- Exportação de relatórios (Edge Function) para seguradoras e envio de newsletter.

6.6 Notificações: Emails e SMS

As notificações são tratadas de forma assíncrona por Edge Functions em Deno. O paciente controla os canais no seu perfil através da tabela user_preferences (email_notifications, sms_notifications, push_notifications, reminder_24h, newsletter_optin).

- Email (Gmail SMTP via denomailer): cancelamento de consulta, novo documento disponível, lembretes 24 horas antes e newsletter semanal.
- SMS (Twilio REST API): quando um documento é carregado e o paciente tem as notificações por SMS ativas e número de telefone, é enviada uma mensagem além do email.
- Push e lembrete 24h: preferências do utilizador que condicionam o envio; o push é sincronizado com email e SMS.

6.7 Edge Functions e Funcionalidades Assíncronas

Edge Function	Responsabilidade
notify-cancellation	Email ao paciente quando uma consulta é cancelada
notify-document	Email e SMS (Twilio) quando um documento é carregado
send-reminders	Email de lembrete 24 horas antes das consultas do dia seguinte
send-newsletter	Envio da newsletter por Gmail SMTP aos subscritores
generate-invoice	Geração automática de fatura em PDF ao concluir a consulta
export-doctor-report	Exportação de relatórios de atos médicos a partir da BD

Outras funcionalidades transversais: registo de auditoria (audit log) das interações dos utilizadores, armazenamento de documentos em Supabase Storage com URLs assinados, e geração de faturas associadas a cada consulta concluída.

7. Arquitetura e Design

A Saúde Prime++ segue um modelo cliente-cloud: uma única base de código Flutter corre em Android, iOS e Web, enquanto toda a persistência, autenticação e lógica de backend ficam no Supabase. O controlo de acesso é feito na base de dados via RLS, sem servidor de aplicação dedicado.

7.1 Vista Lógica (Diagrama de Pacotes)

O sistema organiza-se em três camadas: a camada UI, com cinco pacotes Flutter (um por portal); a camada Core, que expõe o singleton do cliente Supabase partilhado por todos os portais; e a camada Backend (Supabase Cloud), com Auth, PostgreSQL e RLS, Storage e as Edge Functions.

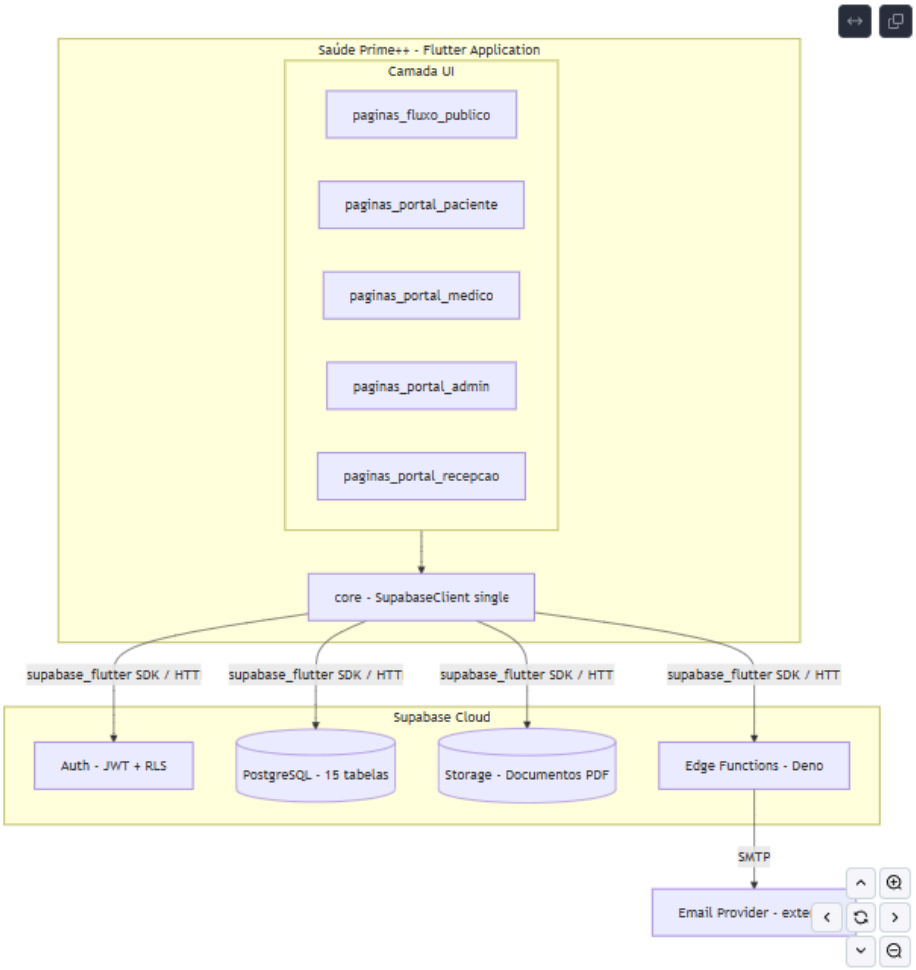


Figura 3: Vista Lógica (camadas e pacotes)

7.2 Vista de Implementação

A app Flutter compilada é um único binário que incorpora os cinco portais e comunica com o Supabase exclusivamente pelo SDK supabase_flutter (v2.12.4). Não existe servidor de aplicação: a lógica de negócio fica no cliente ou nas Edge Functions. O main.dart inicializa o Supabase e encaminha o utilizador para o portal correto com base no role.

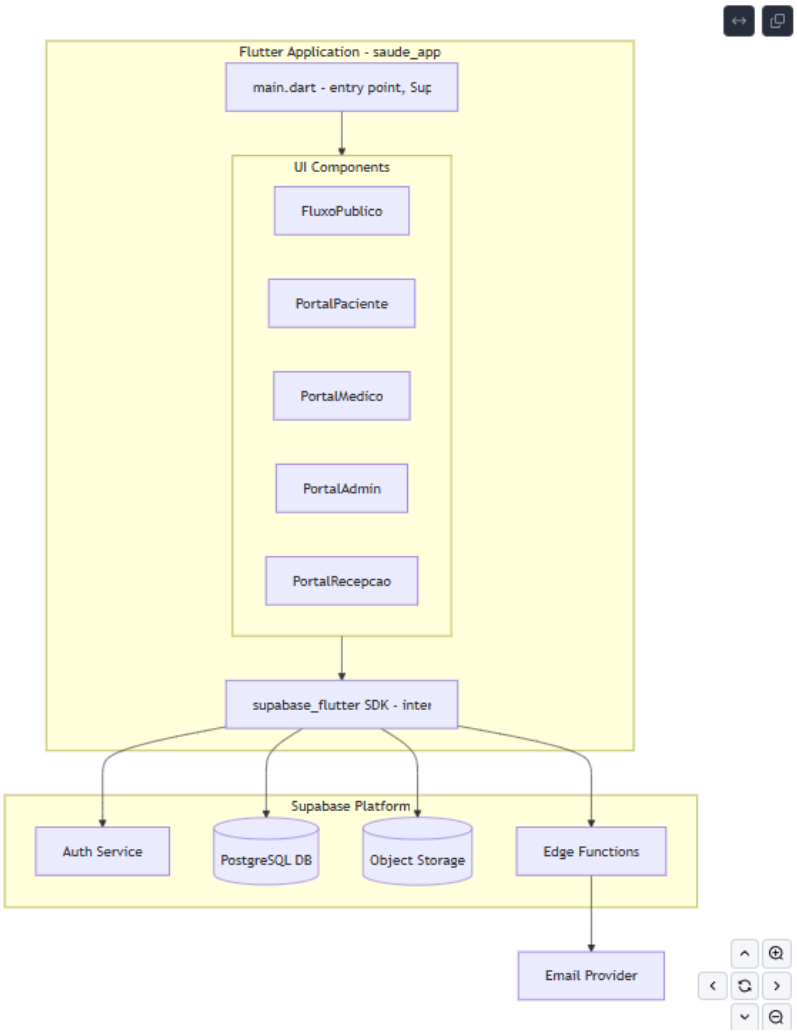


Figura 4: Vista de Implementação (componentes e interfaces)

7.3 Vista de Deployment

Três nós de deployment: o dispositivo do utilizador (Android, iOS ou browser), a Supabase Cloud (GoTrue para Auth, PostgreSQL com RLS, Object Storage e runtime Deno para as Edge Functions) e serviços externos (Gmail SMTP para email e Twilio para SMS).

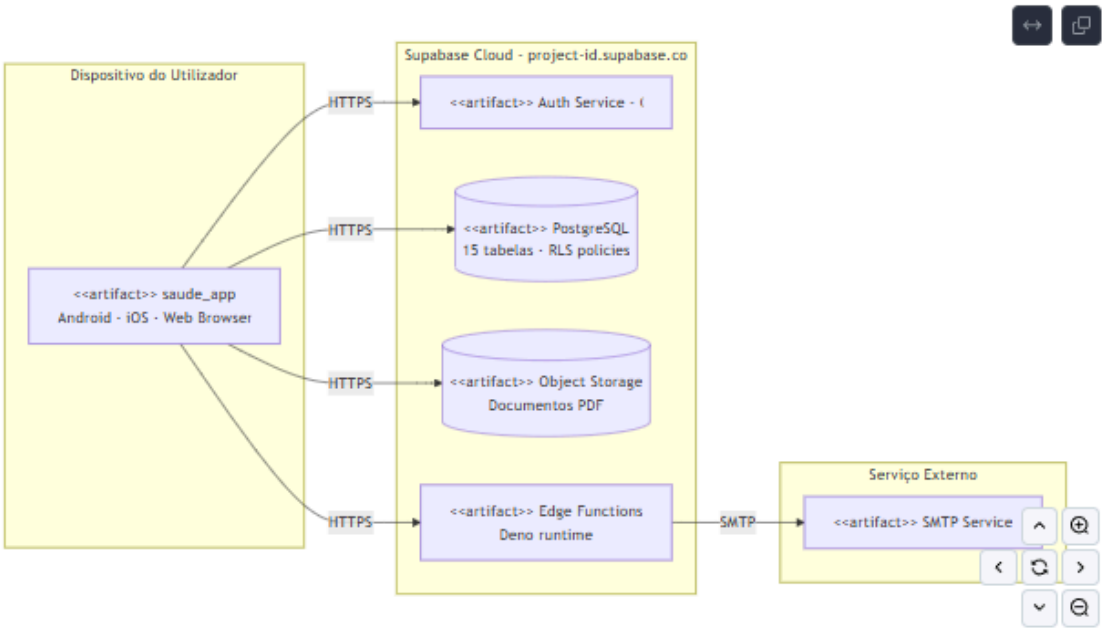


Figura 5: Vista de Deployment (nós cloud)

7.4 Stack Tecnológica

Camada	Tecnologia
Frontend / Cliente	Flutter (Dart): Android, iOS e Web a partir de uma base de código
SDK de integração	supabase_flutter v2.12.4 (REST + WebSocket)
Autenticação	Supabase Auth (GoTrue): JWT, sessão e roles
Base de dados	PostgreSQL (15 tabelas, 25 migrações) com RLS
Armazenamento	Supabase Storage: PDFs clínicos com URLs assinados
Lógica assíncrona	6 Edge Functions em Deno
Serviços externos	Gmail SMTP (email) e Twilio (SMS)

8. Evolução por Versões e Incrementos

O produto foi construído de forma incremental, com cada versão a corresponder a um incremento de produto entregue e a acrescentar uma fatia vertical de valor.

Versão	Incremento	Foco	Ficheiros Dart
v0.0	#0	Protótipo vertical: valida a stack Flutter e Supabase (Landing + Login)	6
v1.0	#1	Frontend completo dos 5 portais (dados simulados)	54
v2.0	#2	Integração de backend: auth real, JWT e RLS	66
v3.0	#3	Edge Functions, notificações, faturas e testes (final)	70

O protótipo vertical (v0.0) implementou a Landing Page e o Login com o cliente Supabase inicializado, validando a comunicação Flutter e Supabase antes de construir a aplicação completa. A partir dessa base de 6 ficheiros foram construídos os 70 ficheiros da versão final.

9. Processo, Iterações e Uso do Board

O trabalho foi organizado em três iterações (mais o incremento zero), cada uma encerrada com uma retrospetiva e a atualização do board Kanban, com o backlog estimado, atribuído e monitorizado.

9.1 Iteração 1: Frontend e Fundações

Todo o frontend foi construído e a base de várias user stories criada, com boa distribuição de tarefas e entrega no prazo. A principal dúvida em aberto era como implementar a base de dados.

9.2 Iteração 2: Integração de Backend

Integração do Supabase com autenticação real e encaminhamento por role nos quatro portais; 6 user stories ligadas à base de dados real. A melhorar: substituir os ecrãs ainda com dados mock e escrever testes a par do desenvolvimento.

9.3 Iteração 3: Edge Functions, Notificações e Testes

Fecho das user stories restantes, incluindo todas as Must Have. As Edge Functions para funcionalidades assíncronas (notificações por email e SMS,

newsletters, lembretes, faturas e relatórios) foram o maior desafio técnico do projeto. Foram adicionados os testes automatizados nos quatro portais.

10. Testes e Qualidade

A estratégia de qualidade combinou testes de aceitação especificados em Gherkin (Given/When/Then), derivados das user stories, com testes unitários automatizados sobre a lógica de negócio. No total, a versão final inclui 21 ficheiros de teste e 286 casos de teste, distribuídos pelos quatro portais.

Portal	Ficheiros de teste	Exemplos de áreas cobertas
Admin	7	Permissões, lista e criação de utilizadores, dashboard, agendas, exportação
Médico	6	Bloqueio de horário, marcar e registar consulta, envio de documentos, perfil
Paciente	4	Dashboard, marcação de consulta, as minhas consultas, perfil e preferências
Receção	4	Marcar/alterar/cancelar consulta, dashboard, tempo de espera e faturas

10.1 Abordagem

Os testes unitários replicam, em funções puras, a lógica de negócio das páginas (por exemplo, a validação de intervalos de bloqueio de horário ou a conversão de datas para UTC), permitindo testar as regras sem levantar um ambiente completo com base de dados real. Os testes de aceitação documentam o comportamento esperado de cada user story em estilo Gherkin.

10.2 Exemplos de Testes de Aceitação

US	Cenário	Resultado esperado
US1	Registo com sucesso	NIF de 9 dígitos e email único: conta criada e confirmação enviada
US6	Bloqueio de agenda	Dia indisponível: horário aparece ocupado ao paciente
US7	Marcação simultânea	Primeiro confirma: segundo recebe aviso de indisponível
US9	Download de fatura PDF	Consulta paga: PDF com dados fiscais corretos
US14	Cancelamento fora do prazo	Faltam 12h: cancelamento impedido pela regra das 24h
US15	Notificação de exame	Novo PDF associado: notificação sem revelar dados sensíveis

11. Fluxo de Trabalho no GitHub

O desenvolvimento seguiu um fluxo baseado em branches de funcionalidade, com integração na main através de merges e pull requests, e marcação de cada release com uma tag.

- 276 commits com mensagens descritivas (por exemplo, feat, fix, docs) e 33 merges.
- 36 feature branches com nomes claros (feature/autenticacao, feature/newsletter, feature/portal_medico_db, feature/reset_password, entre outras).
- Pull requests revistos e integrados, como o #21 (reset password), #22 (newsletter) e #24 (faturas e recibos).
- 4 tags de release (v0.0, v1.0, v2.0, v3.0), uma por incremento de produto.
- Contribuições registadas dos cinco membros da equipa ao longo das três iterações.

12. Dificuldades e Como as superámos

Ao longo das três iterações enfrentamos vários obstáculos técnicos e de organização. Esta secção descreve os principais e a forma como os ultrapassámos, com base nas retrospetivas de cada sprint e no que o histórico de commits revela.

12.1 Não saber por onde começar a base de dados

Dificuldade: no fim da Iteração 1 tínhamos todo o frontend, mas a maior dúvida em aberto era como implementar a base de dados e ligá-la à aplicação. Era o principal risco do projeto.

Como superámos: começámos por um protótipo vertical (v0.0) que validou a comunicação Flutter e Supabase antes de construir tudo, e depois fizemos a integração de forma incremental na v2.0, uma tabela e um ecrã de cada vez, em vez de tentar ligar tudo ao mesmo tempo.

12.2 Políticas de segurança (Row-Level Security)

Dificuldade: garantir que cada role via apenas o que devia revelou-se o maior calcanhar de Aquiles. Houve várias iterações e revisões de políticas RLS por razões de segurança, políticas em falta e casos em que um utilizador não conseguia, ou conseguia em excesso, aceder aos dados.

Como superámos: consolidamos o controlo de acesso em migrações dedicadas às políticas, testámos o acesso role a role e reescrevemos as políticas até cada perfil ficar corretamente isolado. A segurança passou a estar na própria base de dados, sem depender do cliente.

12.3 Migrações e a passagem de dados mock para dados reais

Dificuldade: muitos ecrãs ficaram com dados simulados durante demasiado tempo, o que obrigou a trabalho adicional ao ligá-los à base de dados. Algumas migrações tiveram de ser revertidas (por exemplo, o bloqueio de horários e o registo de auditoria) por introduzirem problemas.

Como superámos: adotamos migrações versionadas e disciplinadas, revertendo e corrigindo quando necessário, e priorizamos a substituição dos dados mock por queries reais logo no início da iteração seguinte.

12.4 Fila virtual em tempo real

Dificuldade: o estado da fila nem sempre atualizava como esperado, com bugs como a senha não desaparecer depois de cancelar a consulta e o número da fila aparecer errado.

Como superámos: corrigimos a lógica de estado e as queries que alimentavam a fila. Reconhecemos que uma solução totalmente em tempo real, com Supabase Realtime a maior escala, fica como trabalho futuro.

12.5 Testes automáticos para Flutter e Supabase

Dificuldade: não era óbvio como escrever testes fiáveis para uma app Flutter ligada ao Supabase. Os testes de integração exigiam levantar um ambiente com base de dados real, o que era pesado e frágil.

Como superámos: abandonamos o teste de integração e passámos a testes unitários de lógica pura, replicando as funções de validação e de negócio (por exemplo, validação de bloqueio de horário e conversão de datas para UTC). Assim chegámos a 286 casos de teste sem precisar de uma base de dados a correr.

12.6 Notificações assíncronas e segredos (email e SMS)

Dificuldade: as funcionalidades assíncronas foram o maior desafio técnico do projeto. Integrar o envio de email (Gmail SMTP) e de SMS (Twilio), e gerir as credenciais em segurança, não foi trivial.

Como superámos: isolámos esta lógica em Edge Functions (Deno), mantivemos os segredos fora dos assets da aplicação (configuração de .env) e testámos o envio. As preferências do utilizador passaram a controlar cada canal de notificação.

12.7 Trabalho em equipa numa única base de código

Dificuldade: cinco pessoas a trabalhar em quatro portais sobre o mesmo repositório, com user stories grandes (L/XL) difíceis de monitorizar no board.

Como superámos: usamos branches por funcionalidade, integramos via pull requests e marcamos cada release com uma tag. A lição que fica é partir as user stories grandes em tarefas mais pequenas para ter melhor visibilidade do progresso.

13. Uso de Inteligência Artificial

Consideramos que a utilização da inteligência artificial foi importante para o desenvolvimento da nossa aplicação e para o esclarecimento de dúvidas. Foi-nos útil a geração de código das migrations, para adicionar parâmetros à database, e nas nossas edge functions. Também foi importante para alguns casos isolados com o objetivo de corrigir erros e debugging.

14. Contribuição do Grupo

Aluno	Contribuição	Nota Pretendida
Felipe Castilho	20%	20
Juliana Moreira	20%	20
Marta Vieira	20%	20
Ricardo Dias	20%	20
Rodrigo Esteves	20%	20

15. Conclusão

A Saúde Prime++ cumpriu o objetivo a que se propôs: entregar uma aplicação de gestão clínica multiperfil, funcional e pronta para produção, com todas as user stories implementadas e ligadas a um backend real. Todos os entregáveis da unidade curricular foram concluídos e versionados, e o produto integra notificações por email e SMS, faturas automáticas, fila virtual e exportação de relatórios.

A adoção de Flutter com Supabase permitiu suportar Android, iOS e Web a partir de uma única base de código, e o recurso a RLS garantiu controlo de acesso robusto sem a complexidade de um servidor dedicado. O percurso incremental, do protótipo vertical à versão final, demonstrou o valor de validar cedo as decisões arquiteturais e de entregar valor de forma contínua.